

Employee Management Platform

Architecture & Deployment Guide

Document Overview

This document describes the architecture, infrastructure, deployment strategy, and operational approach for the Employee Management Platform.

The platform demonstrates modern DevOps practices by provisioning cloud infrastructure using Terraform, automating deployments through GitHub Actions, and running containerized workloads on Amazon ECS Fargate.

The objective is to provide a repeatable, production-inspired deployment model that emphasizes automation, scalability, security, and maintainability.

Solution Overview

The platform consists of two independent application services:

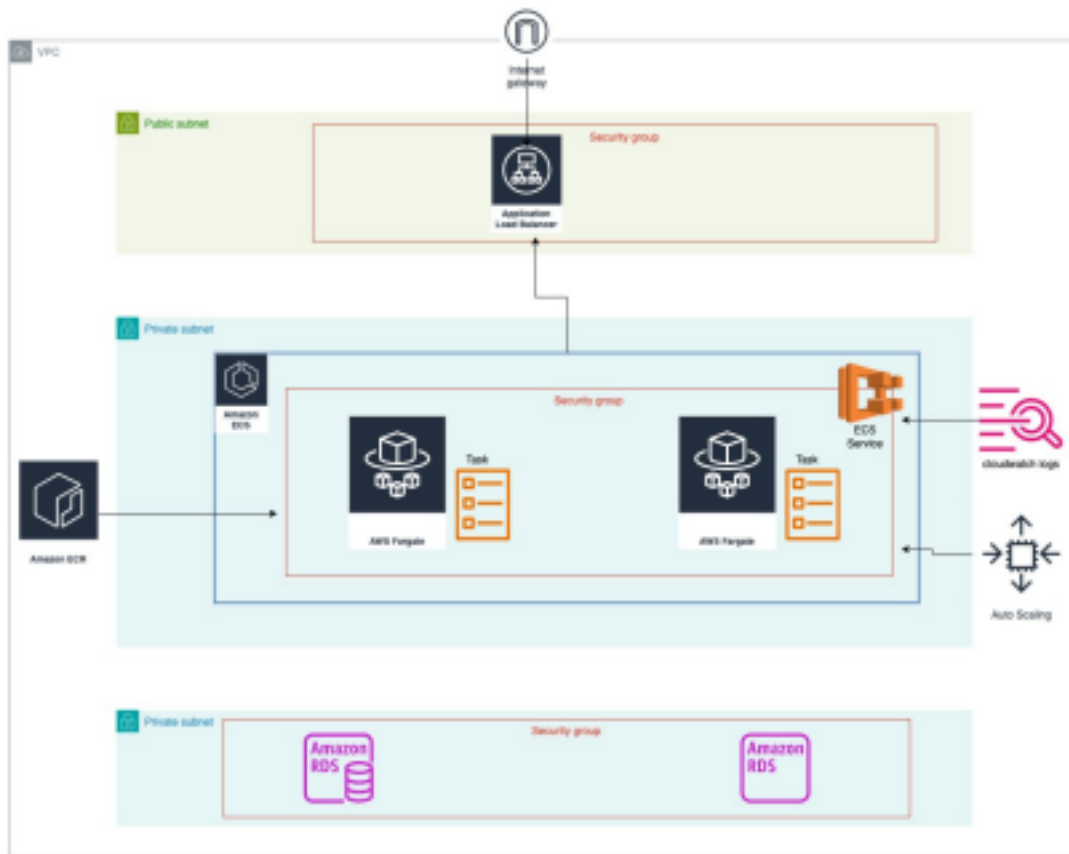
- **Frontend** – React application
- **Backend** – Spring Boot (Java) REST API

Both services are containerized with Docker and deployed independently on Amazon ECS Fargate.

Infrastructure provisioning is fully automated using Terraform, while GitHub Actions manages continuous integration and continuous deployment.

The application persists data in Amazon RDS and exposes public traffic through an Application Load Balancer.

High-Level Cloud Architecture Diagram



The solution consists of the following AWS services:

- Amazon ECS (Fargate)
- Amazon Elastic Container Registry (ECR)
- Amazon RDS (MySQL)
- Application Load Balancer (ALB)
- Amazon VPC
- Amazon Route 53
- Amazon CloudWatch
- AWS CloudTrail

- AWS Certificate Manager (ACM)
- GitHub Actions
- Terraform

Infrastructure Design

Compute Layer

Application workloads are deployed as Docker containers on **Amazon ECS Fargate**.

The frontend and backend run as independent ECS services, allowing each component to be deployed and scaled independently.

Design Decisions

- Serverless container management
- No EC2 instance administration
- Independent service deployments
- Simplified scaling
- Reduced operational overhead

Networking

The platform is deployed inside a dedicated Amazon VPC.

Public Subnets

Public subnets host:

- Application Load Balancer
- NAT Gateway

Private Subnets

Private subnets host:

- ECS Tasks
- Amazon RDS

Keeping application workloads inside private subnets reduces exposure while allowing outbound internet connectivity through the NAT Gateway.

Database Layer

Amazon RDS (MySQL) provides the persistent storage layer.

Why Amazon RDS?

Amazon RDS was selected because it provides:

- Managed backups
- Automated patching
- High availability support
- Multi-AZ capability
- Operational simplicity

The database is deployed within private subnets and is not directly accessible from the public internet.

Load Balancing

Application traffic enters through an **Application Load Balancer (ALB)**.

Responsibilities include:

- HTTPS termination
- Request routing
- Health checks
- Target group management

Separate target groups are configured for the frontend and backend services.

SSL certificates are managed using AWS Certificate Manager (ACM).

DNS

Amazon Route 53 manages DNS records for the platform.

Route 53 maps custom domain names to the Application Load Balancer, providing a single public entry point for client requests.

Identity & Access Management

IAM roles are provisioned using the principle of least privilege.

Separate permissions are assigned for:

- ECS Tasks
- GitHub Actions deployments
- Terraform execution
- Container registry access

Monitoring & Auditing

Operational visibility is provided through:

Amazon CloudWatch

- Container logs
- Metrics
- ECS service monitoring
- Application health

AWS CloudTrail

CloudTrail records infrastructure API activity, providing an audit trail for operational and security events.

CI/CD Architecture

Deployment automation is implemented using GitHub Actions.

Two independent pipelines are maintained.

Infrastructure Pipeline

Purpose:

Provision and manage AWS infrastructure.

Workflow

1. Checkout repository
2. Configure Terraform
3. Initialize Terraform
4. Validate configuration
5. Generate execution plan
6. Apply infrastructure changes
7. Optional destroy workflow

Infrastructure changes are fully version controlled through Terraform.

Application Deployment Pipeline

Application deployments follow the workflow below.

Frontend

1. Build React application
2. Build Docker image
3. Push image to Amazon ECR
4. Update ECS Task Definition
5. Deploy to ECS Service

Backend

1. Build Spring Boot application
2. Build Docker image
3. Push image to Amazon ECR

4. Update ECS Task Definition

5. Deploy to ECS Service

This approach enables independent deployments of frontend and backend services without affecting the entire platform.

Scalability

The platform is designed to support horizontal scaling.

Amazon ECS Service Auto Scaling automatically adjusts the desired task count based on CloudWatch metrics including:

- CPU Utilization
- Memory Utilization

This allows the application to respond dynamically to changes in workload while optimizing infrastructure costs.

Deployment Workflow

The deployment process follows the sequence below.

1. Provision infrastructure using Terraform.
2. Build application artifacts.
3. Build Docker container images.
4. Push images to Amazon ECR.
5. Update ECS task definitions.
6. Deploy services to Amazon ECS.
7. Route traffic through the Application Load Balancer.
8. Monitor platform health using CloudWatch.

Local Deployment

Prerequisites

- AWS Account
- Terraform CLI
- AWS CLI
- Docker
- Git

Infrastructure Deployment

terraform init

terraform validate

terraform plan

terraform apply

Application Deployment

Push changes to the deployment branch.

GitHub Actions automatically:

- Builds the application
- Builds Docker images
- Pushes images to Amazon ECR
- Deploys updated services to Amazon ECS

Production Considerations

This implementation demonstrates several production-oriented engineering practices:

- Infrastructure as Code
- Immutable container deployments
- Automated CI/CD

- Network segmentation
- HTTPS termination
- Managed database services
- Centralized monitoring
- Infrastructure auditing
- Auto Scaling
- Least privilege IAM

Username Password Log In

Hello, user.

Log Out Create

Employees - Page 1 of 5

2

First Name	Last Name	Description	Manager		
Frodo	Baggins	ring bearer	greg	Not Your Employee	Delete
Bilbo	Baggins	burglar	greg	Not Your Employee	Delete

<< > >>

Amazon Elastic Container Service > Clusters > java-react-api > Services

Last updated 15 December 2024 at 08:02 (UTC+1:00) [Update cluster](#) [Delete cluster](#)

Cluster overview

ARN: [arn:aws:ecs:eu-west-1:123456789012:cluster/java-react-api](#) Status: Active

CloudWatch monitoring: [Container Insights](#) [View in CloudWatch](#)

Registered container instances: 0

Services: Draining: 0 Active: 2 Pending: 0 Running: 2

Encryption: Managed storage: 0 Fargate ephemeral storage: 0

[Services](#) [Tasks](#) [Infrastructure](#) [Metrics](#) [Scheduled tasks](#) [Tags](#)

Services (2)

Filter services by value: Filter launch type: Any launch type Filter service type: Any service type

Service name	ARN	Status	Service type	Deployments and tasks	Last deployment	Task definition	La...
backend-service	arn:aws:ecs:eu-west-1:123456789012:cluster/java-react-api:service/backend-service	Active	REPLICA	1/1 tasks running	Completed	backend-task-26	FAR...
frontend-service	arn:aws:ecs:eu-west-1:123456789012:cluster/java-react-api:service/frontend-service	Active	REPLICA	1/1 tasks running	Completed	frontend-task-31	FAR...

Amazon Elastic Container Service > Clusters > java-react-api > Services > backend-service > Service auto scaling > View

Last updated 15 December 2024 at 08:04 (UTC+1:00) [Update service](#) [Delete service](#)

Service overview

Status: Active Tasks (1 Desired): 0 pending | 1 running Task definition: revision [backend-task-26](#) Deployment status: Success

[Health and metrics](#) [Tasks](#) [Logs](#) [Deployments](#) [Events](#) [Configuration and networking](#) [Service auto scaling](#) [Tags](#)

How to use service auto scaling

Task count: Suspend/Unsuspend scaling Edit Set the number of tasks

Number of tasks to run for backend-service: 2 - 4

[Scaling policies](#) [Scheduled actions](#) [Recommendations](#)

Scaling policies (2)

Find a scaling policy:

Name	Type	Status	Scaling mechanism	Target metric	Actions
backend_scale_down	Step Scaling	Active	Remove capacity based on threshold	1 step(s)	View
backend_scale_up	Step Scaling	Active	Add capacity based on threshold	1 step(s)	View

[Scaling activities](#)

Amazon Elastic Container Service > Clusters > java-react-api > Services > frontend-service > Service auto scaling > View

Amazon Elastic Container Service

Clusters Updated
Namespaces
Task definitions
Account settings Updated

Install AWS Copilot
Amazon ECR
Repositories
AWS Batch
Documentation
Discover products
Subscriptions
Tell us what you think

frontend-service Info
Last updated 15 December 2024 at 08:05 (UTC+1:00) Update service Delete service

Service overview Info
Status Active
Tasks (1 Desired) 0 pending 1 running
Task definition: revision frontend-task-31
Deployment status Success

Health and metrics Tasks Logs Deployments New Events Configuration and networking Service auto scaling New Tags

► How to use service auto scaling

Task count
Suspend/Unsuspend scaling Edit Set the number of tasks
Number of tasks to run for frontend-service 2 - 4

Scaling policies Scheduled actions Recommendations

Scaling policies (2) Info
Find a scaling policy

Name	Type	Status	Scaling mechanism	Target metric	Actions
frontend_scale_down	Step Scaling	Active	Remove capacity based on threshold	1 step(s)	View
frontend_scale_up	Step Scaling	Active	Add capacity based on threshold	1 step(s)	View

Scaling activities

© 2024, Amazon Web Services, Inc. or its affiliates. Privacy Terms Cookie preferences

RDS > Databases > testclouddb

Amazon RDS

Dashboard
Databases
Query Editor
Performance insights
Snapshots
Exports in Amazon S3
Automated backups
Reserved Instances
Proxies
Subnet groups
Parameter groups
Option groups
Custom engine versions
Zero-ETL integrations New
Events
Event subscriptions
Recommendations
Certificate update

testclouddb
Modify Actions

Summary
DB Identifier testclouddb
CPU 2.98%
Status Available
Class db.t3.micro
Role Instance
Current activity 10 Connections
Engine MySQL Community
Region & AZ eu-west-1a
Recommendations

Connectivity & security Monitoring Logs & events Configuration Zero-ETL integrations Maintenance & backups Data migrations - new Tags Recommendations

Connectivity & security

Endpoint & port
Endpoint testcloudb-west-1.rds.amazonaws.com
Port 3306

Networking
Availability Zone eu-west-1a
VPC BH VPC (vpc-0e634553807ca5dc0)
Subnet group database-subnets
Subnets subnet-0b88393da594dfcd subnet-095adac30fa5464ee
Network type IPv4

Security
VPC security groups Database Security Group (sg-053b0f49a775860f8) Active
Publicly accessible No
Certificate authority rds-ca-rsa2048-g1
Certificate authority date May 20, 2061, 18:49 (UTC+01:00)
DB instance certificate expiration date December 15, 2025, 07:19 (UTC+01:00)